The OpenMP

What is OpenMP?·

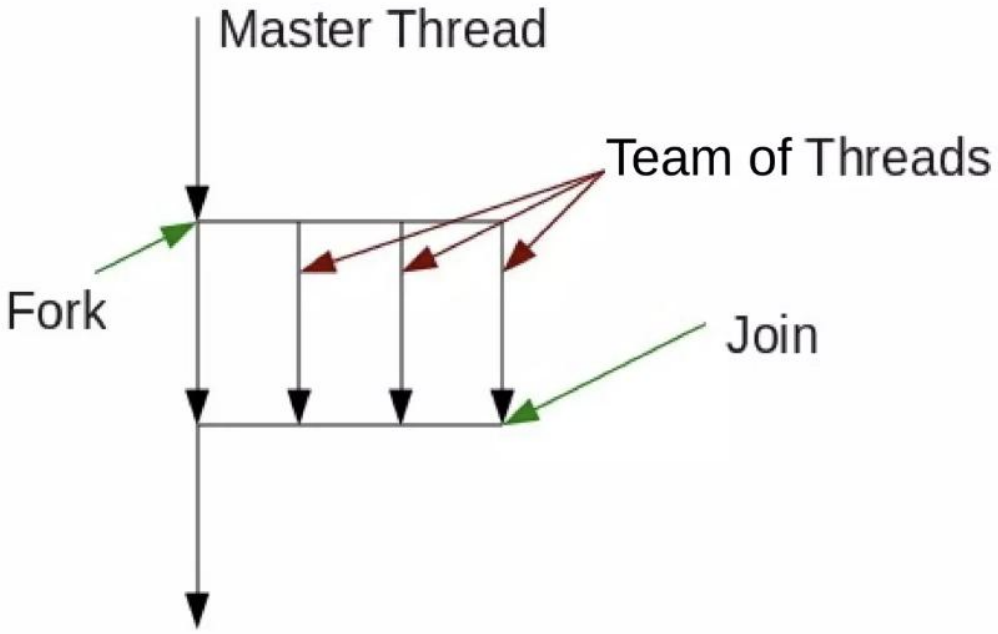
- OpenMP - Open Multi-Processing.
- API for multi-threaded, shared memory parallelism.
- Designed for multi processor/core shared memory machines UMA/NUMA accesses.
- Three primary components are:
  - Compiler directives
  - Runtime library routines
  - Environment variables.
- Supports C/C++, Fortran.

Programming Model

- Thread based parallelism
- Explicit parallelism – programmer has full control
- Fork - Join Model of execution

Each thread is assigned a unique id within team.

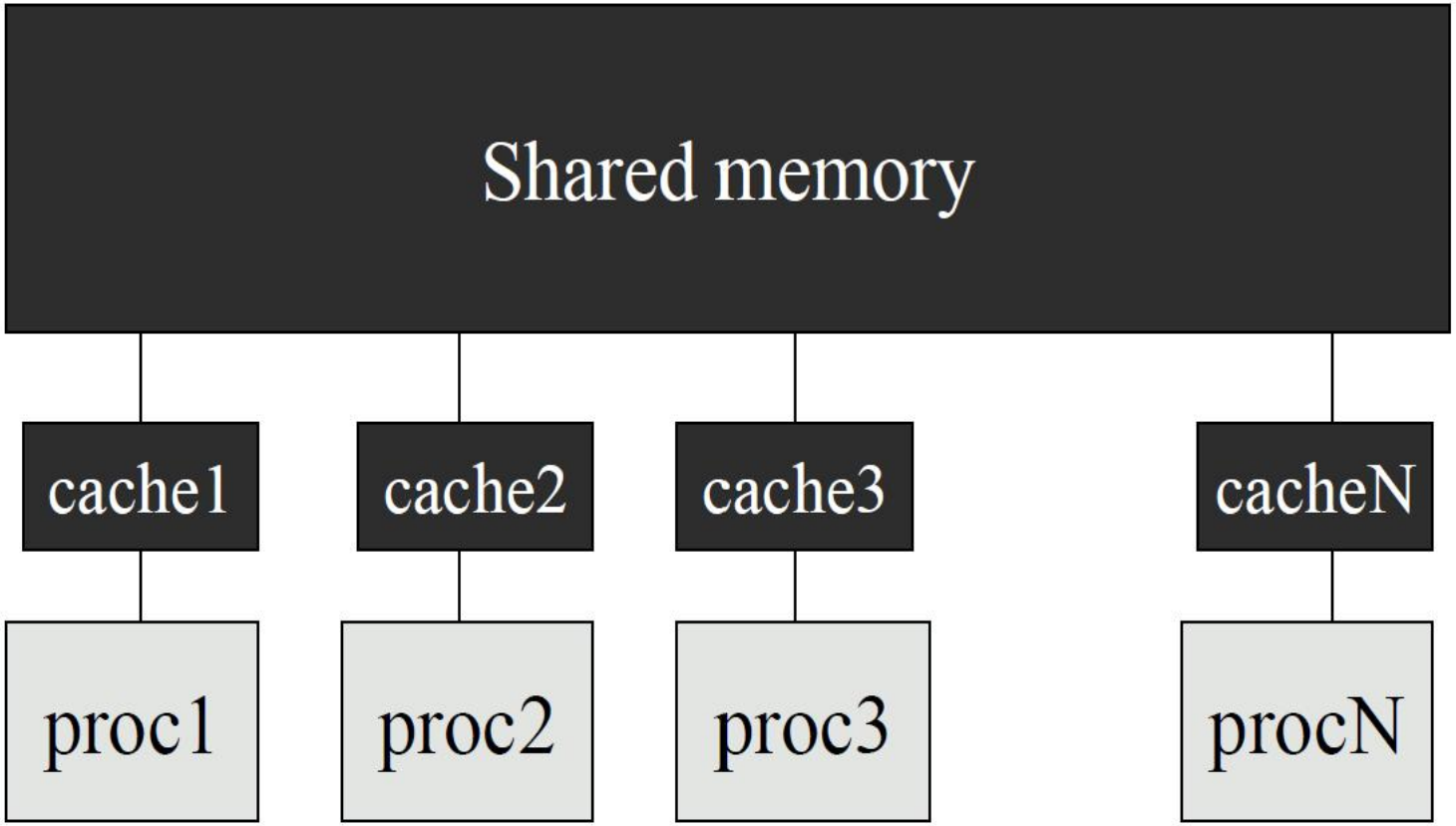
Master thread has id 0.



What are threads?

- Thread: an independent flow of control
  - Runtime entity created to execute sequence of instructions
- Threads require:
  - A program counter
  - A register state
  - An area in memory, including a call stack
  - A thread id
- A process is executed by one or more threads that share:
  - Address space
  - Attributes such as UserID, open files, working directory, etc.

A Shared Memory Architecture



How Can We Exploit Threads?

A thread programming model must provide (at least) the means to:

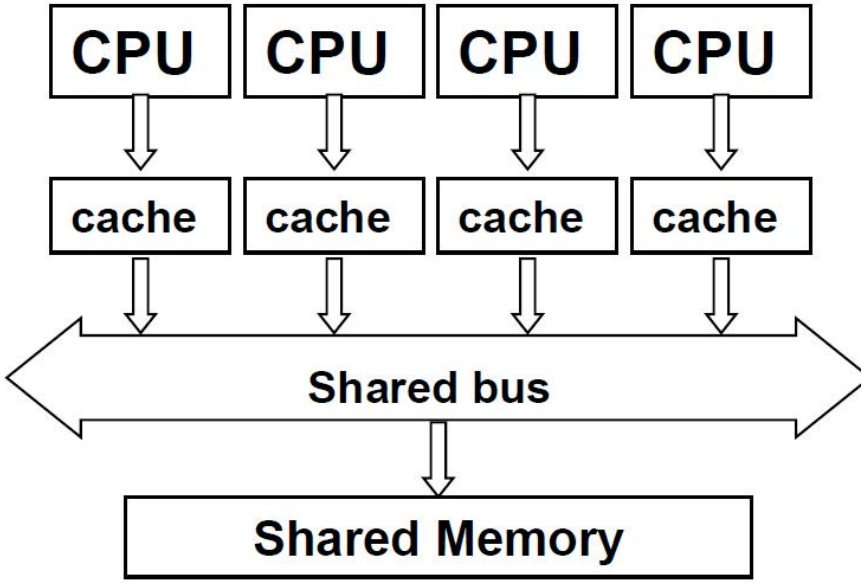
- Create and destroy threads
- Distribute the computation among threads
- Coordinate actions of threads on shared data
- (usually) specify which data is shared and which is private to a thread

How Does OpenMP Enable Us to Exploit Threads?

- OpenMP provides thread programming model at a "high level".
  - The user does not need to specify all the details
    - Especially with respect to the assignment of work to threads
    - Creation of threads
- User makes strategic decisions
- Compiler figures out details
- Alternatives:
  - MPI
  - POSIX thread library is lower level
  - Automatic parallelization is even higher level (user does nothing)
    - But usually successful on simple codes only

Where Does OpenMP Run?

Hardware Platforms	OpenMP support
Shared Memory Systems	Available
Distributed Shared Memory Systems (ccNUMA)	Available
Distributed Memory Systems	Available via Software DSM
Machines with Chip MultiThreading	Available

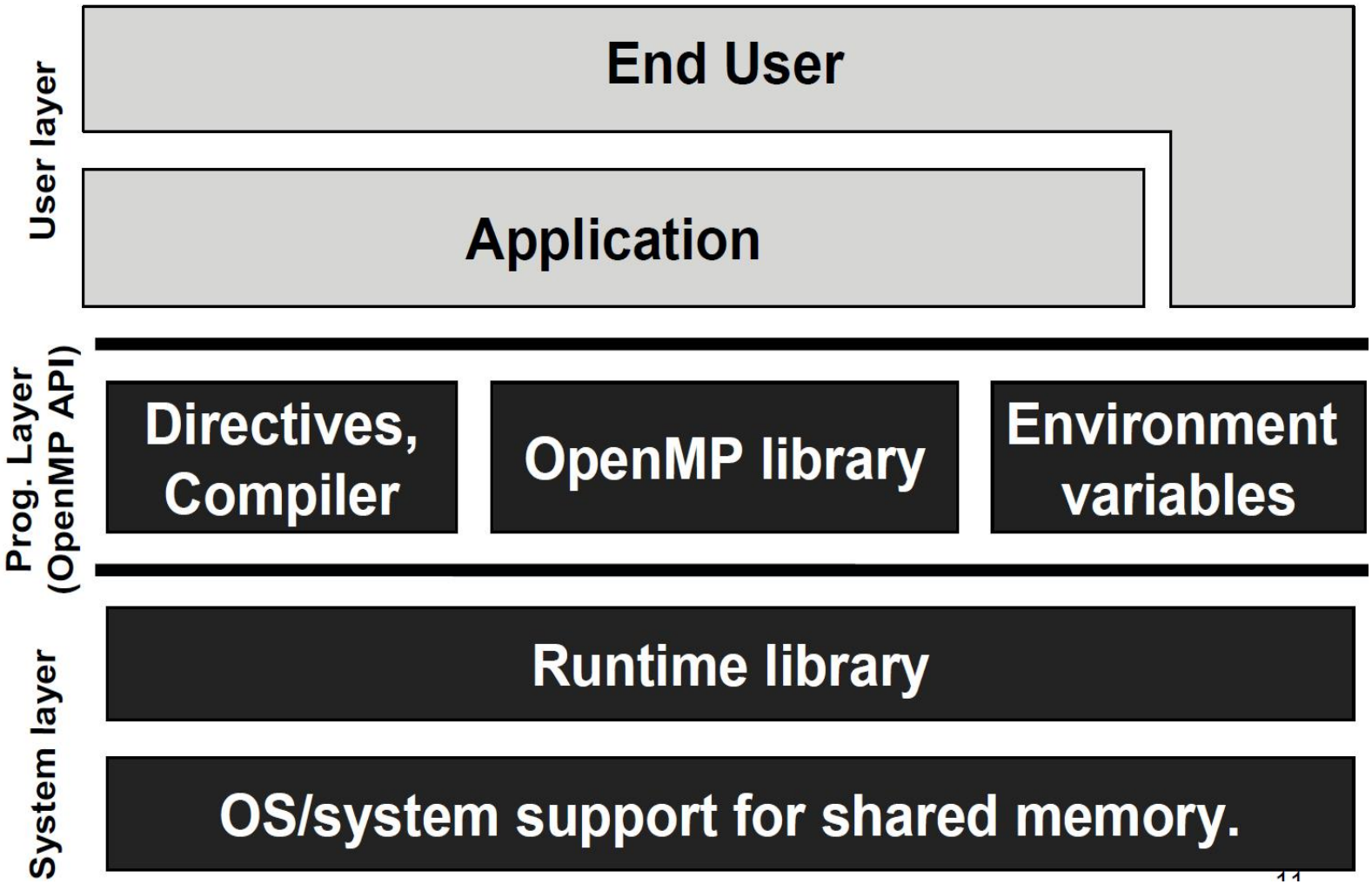


Shared memory architecture

How do threads interact?

- OpenMP is a shared memory model.
  - Threads communicate by sharing variables.
- Unintended sharing of data causes race conditions:
  - race condition: when the program's outcome changes as the threads are scheduled differently.
- To control race conditions:
  - Use synchronization to protect data conflicts.
- Synchronization is expensive so:
  - Change how data is accessed to minimize the need for synchronization.

OpenMP Parallel Computing Solution Stack



OpenMP: Some syntax details to get us started.

- Most of the constructs in OpenMP are compiler directives.

- For C and C++, the directives are pragmas with theform:

```
#pragma omp construct [clause [clause] ... ]
```

- Include file and the OpenMP lib module

```
#include <omp.h>
```

```
use omp_lib
```

OpenMP

Structure block always Start and End with {} bracket

Structured blocks (C/C++)

- **Most OpenMP* constructs apply to structured blocks.**
  - **Structured block: a block with one point of entry at the top and one point of exit at the bottom.**
  - **The only "branches" allowed are STOP statements in Fortran and exit() in C/C++.**

```
#pragma omp parallel
{
        int id = omp_get_thread_num();
more:  res[id] = do_big_job(id);
        if(!conv(res[id]) goto more;
}
   printf(" All done \n");
```

A structured block

```
     if(go_now()) goto more;
#pragma omp parallel
{
        int id = omp_get_thread_num();
more:  res[id] = do_big_job(id);
        if(conv(res[id]) goto done;
        go to more;
}
done:     if(!really_done()) goto more;
```

Not A structured block

OpenMP

Structured Block Boundaries

- In C/C++: a block is a single statement or a group of statements between brackets {}

```
#pragma omp parallel
{
    id = omp_thread_num();
    res(id) = lots_of_work(id);
}
```

```
#pragma omp for
  for(I=0;I<N;I++){
      res[I] = big_calc(I);
      A[I] = B[I] + res[I];
  }
```

OpenMP: Parallel Regions

- You create threads in OpenMP* with the “omp parallel” pragma.
- For example, To create a 4 thread Parallel region:

Each thread executes a copy of the code within the structured block

```
double A[1000];
omp_set_num_threads(4);
#pragma omp parallel
{
        int ID = omp_get_thread_num();
        pooh(ID,A);
}
```

Runtime function to request a certain number of threads

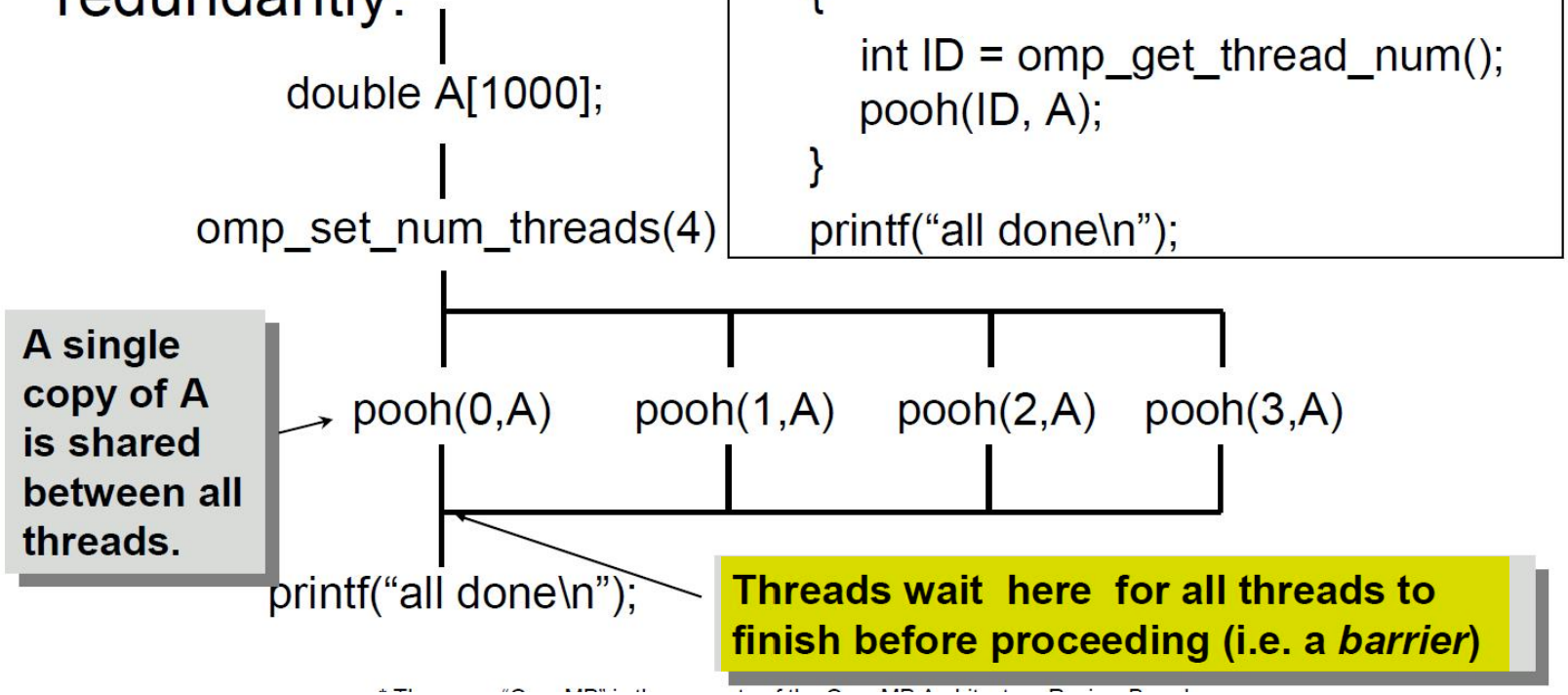
Runtime function returning a thread ID

- **Each thread calls pooh(ID,A) for `ID` = 0 to 3**

OpenMP: Parallel Regions

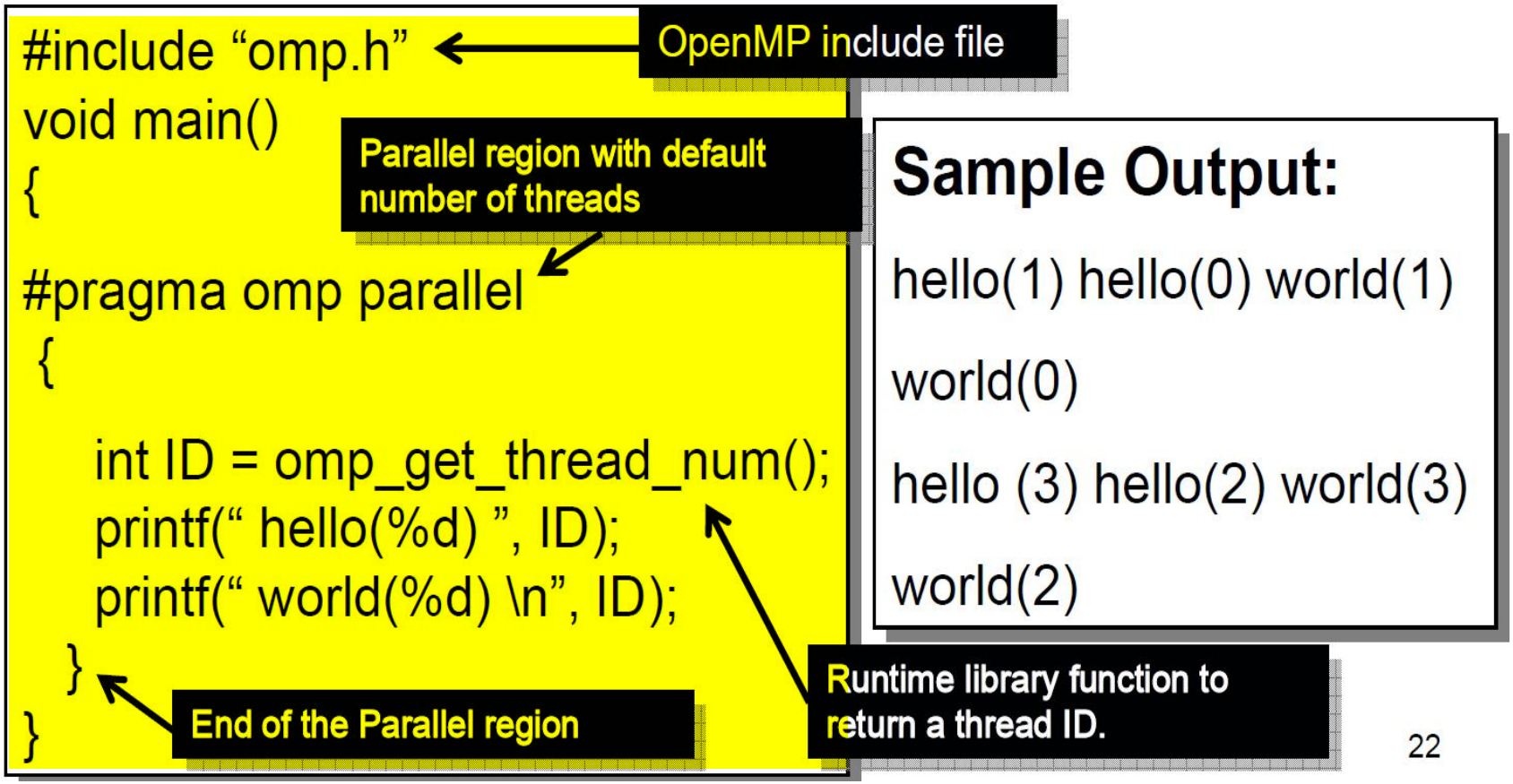
- Each thread executes the same code redundantly.

```
double A[1000];
omp_set_num_threads(4);
#pragma omp parallel
{
    int ID = omp_get_thread_num();
    pooh(ID, A);
}
printf("all done\n");
```



Exercise

Write a multi threaded program where each thread prints "hello world".



Parallel Regions and the “if” clause

Active vs *inactive* parallel regions.

- An optional if clause causes the parallel region to be active only if the logical expression within the clause evaluates to true.
- An if clause that evaluates to false causes the parallel region to be inactive (i.e. executed by a team of size one).

```
double A[N];

#pragma omp parallel if(N>1000)
{
        int ID = omp_get_thread_num();
        pooh(ID,A);
}
```

true false condition, if true then do work